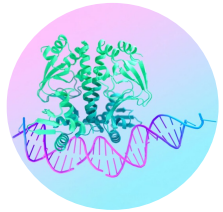
SciLifeLab

AI agent collaboration with the Model Context Protocol (MCP)



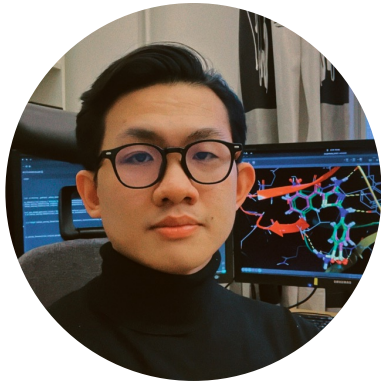




Mahbub Ul Alam
Data Engineer (AI)
SciLifeLab Data Centre



Johan Alfredéen
Data Engineer (AI)
SciLifeLab Data Centre



Dinh Long Huynh
Doctoral Candidate
Uppsala University





02/11/26

Where we are in the day...

This morning: you built an agent (LangGraph)

Now: you use that concept in a reusable research service, through Model Context Protocol (MCP)

By 15:00 you will have

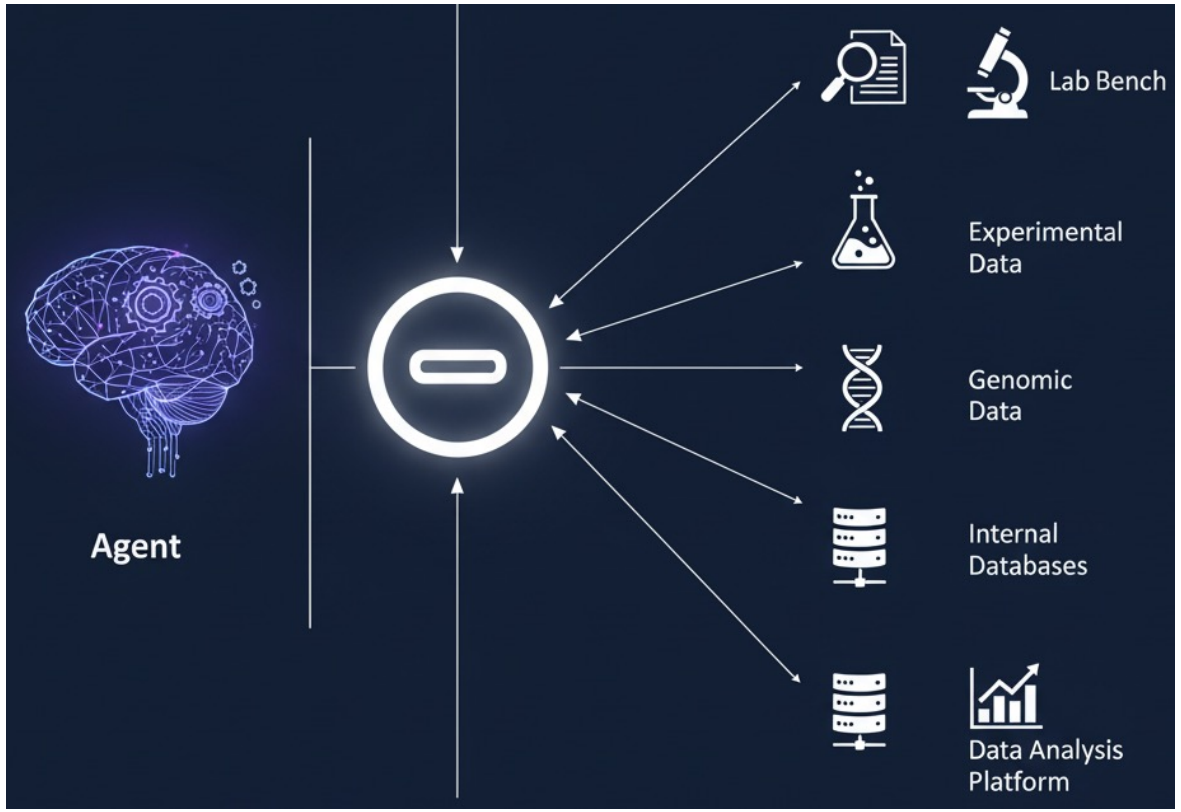
- **Explored MCP:** Seen how AI agents communicate with external tools.
- **Created a tool:** Built a simple tool for searching scientific papers.
- **Tested your tool:** Checked that it works, even without an AI model.
- **Connected your agent:** Given your AI agent access to the literature search tool.
- **Checked the results:** Learned how to identify errors and verify that the AI cites real papers.
- **Added safety checks:** Made sure your agent only performs actions it is allowed to do.

The Real-world challenges: scalability & reinventing the wheel



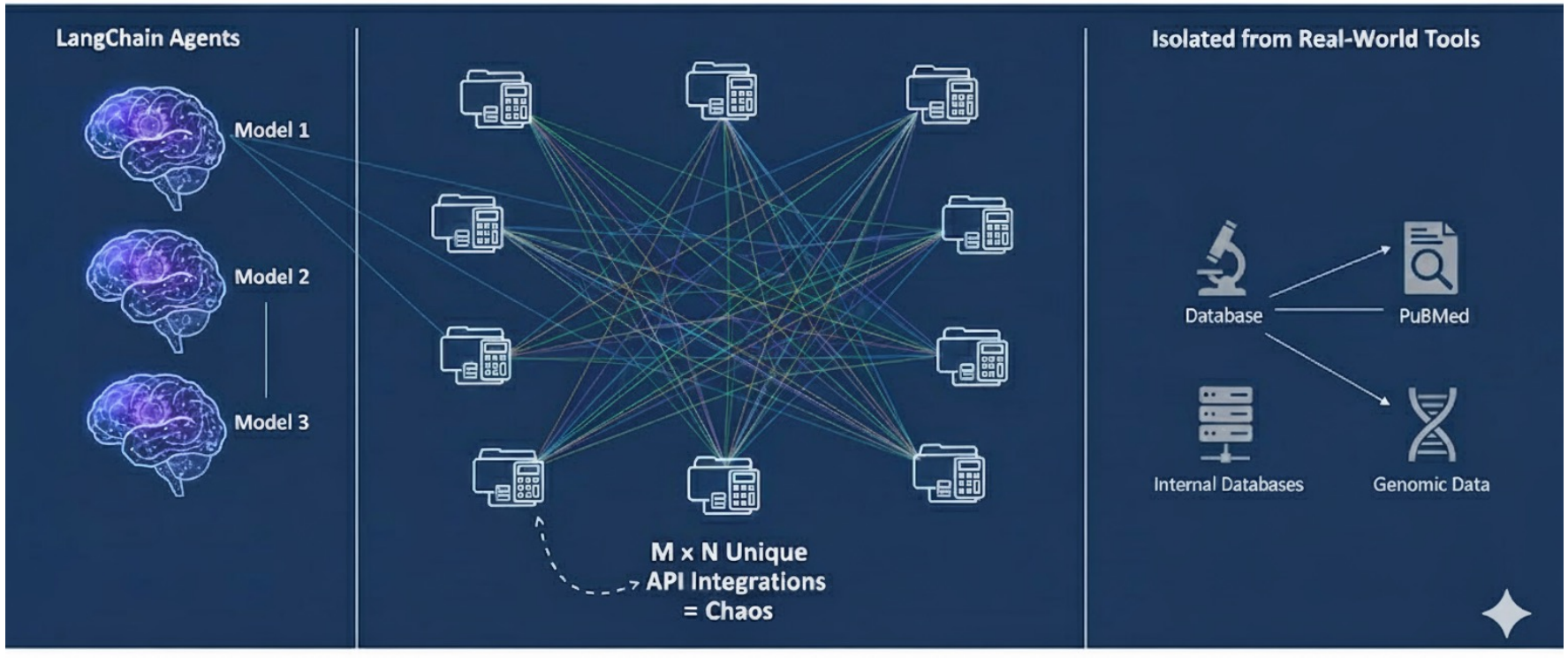
Solution: introducing MCP (Model Context Protocol)!

- LangGraph = **The brain** (reasoning and planning)
- MCP = **The nervous system** (standardized connections)
- **Dynamic tool discovery**: agents find and use tools automatically
- **Unified ecosystem**: teams build and share tools composably



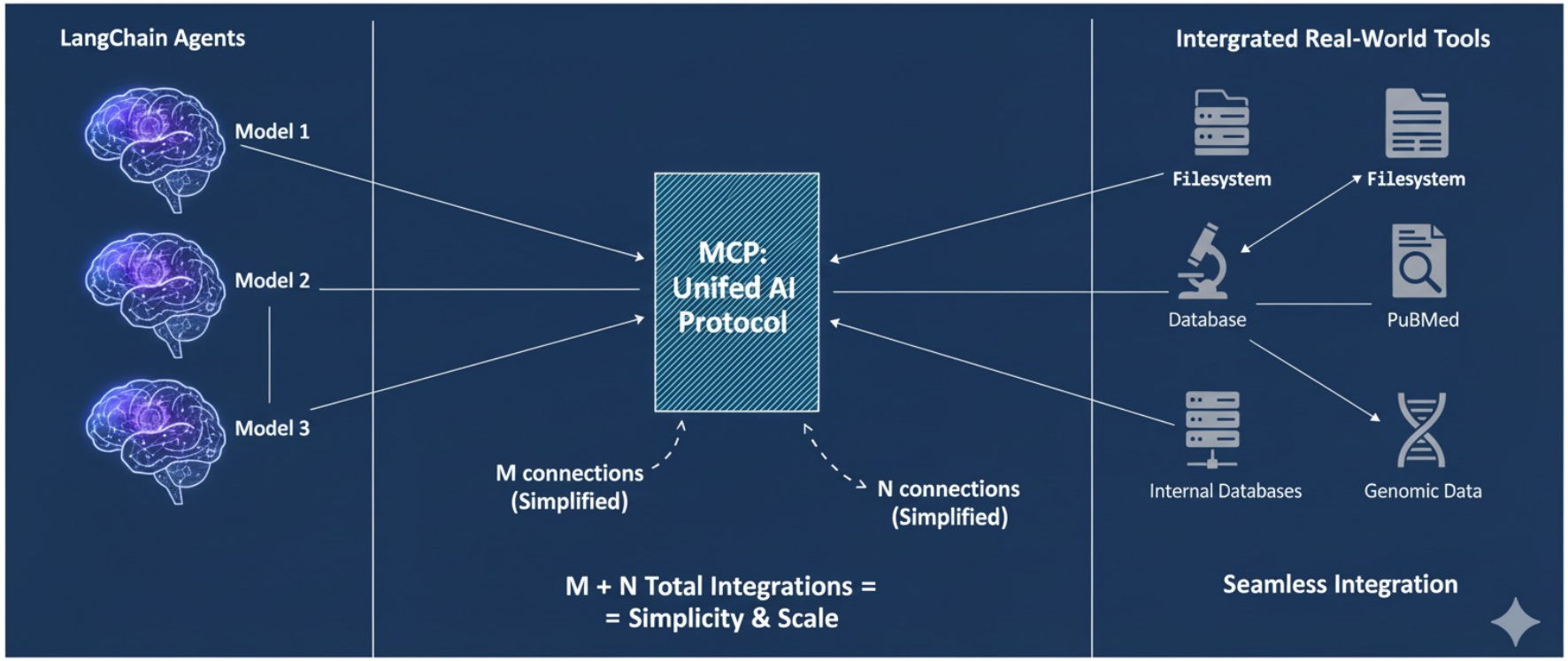
MCP: The **“USB-C for AI”**

The M×N integration problem



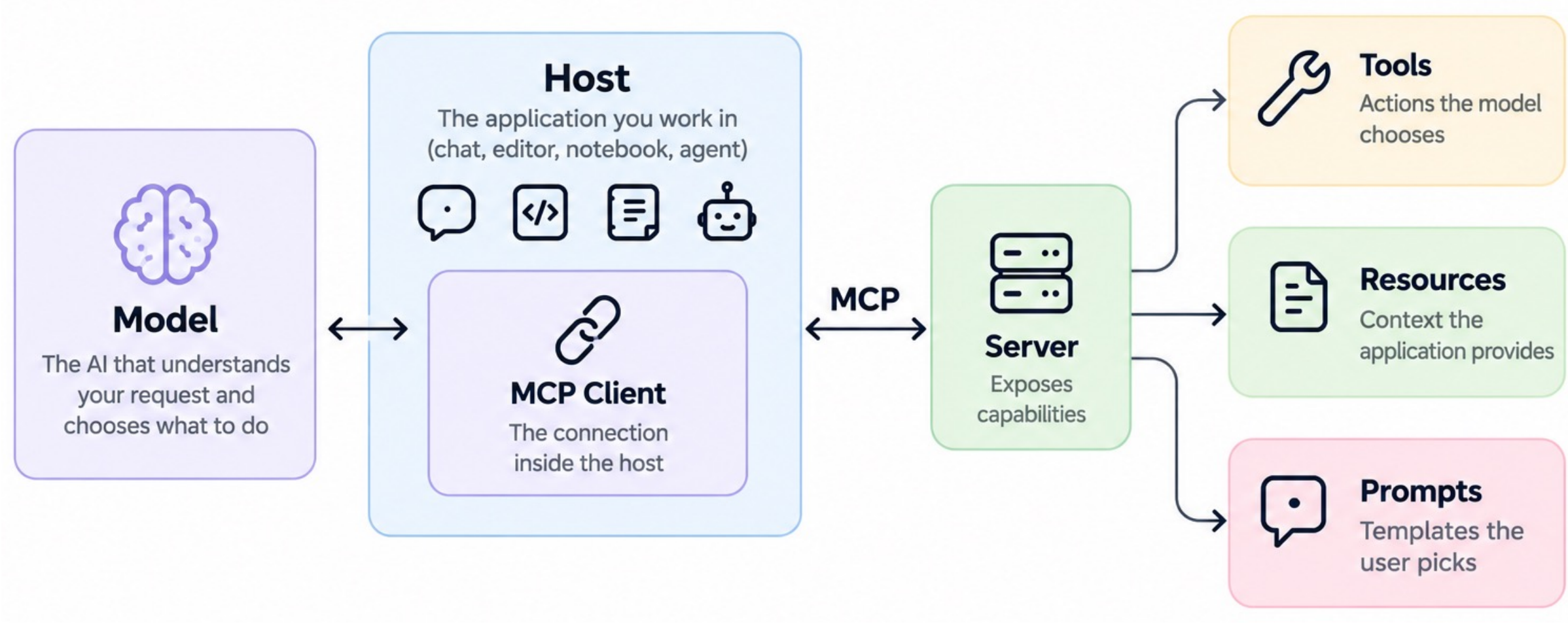
- Many agents and applications, many tools and data sources
- Without a standard, every pair needs its own glue code

With MCP (M+N solution)



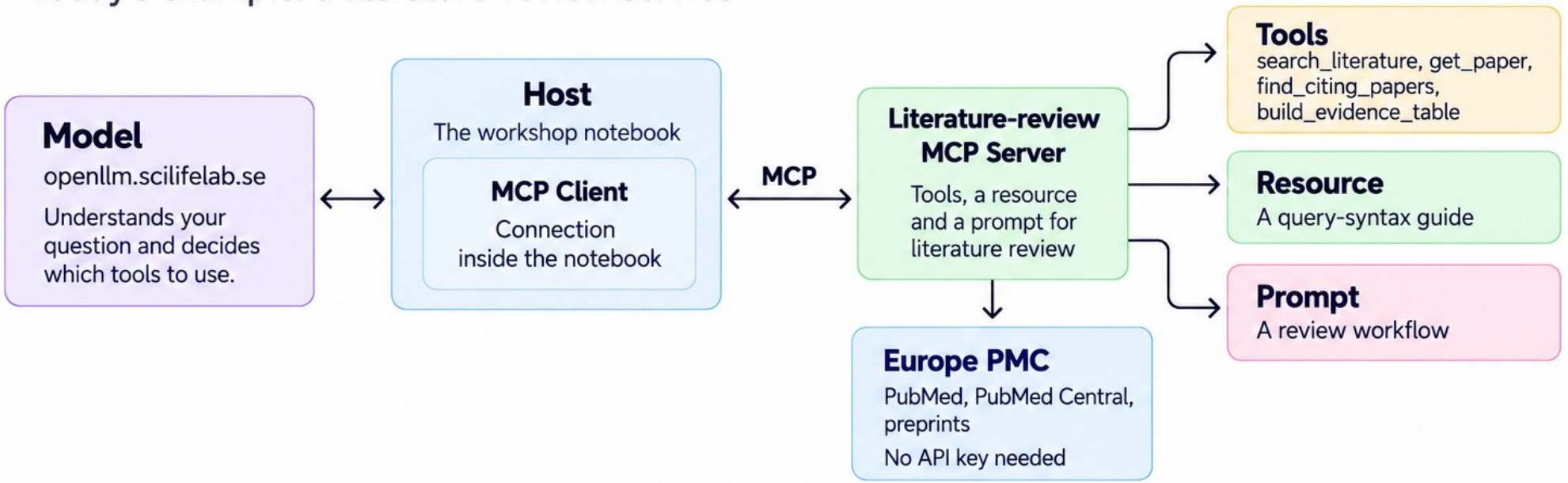
- With MCP, each capability is wrapped once and every host can use it
- Worth it when a capability is reused

MCP in one picture



How the workshop notebook is prepared

Today's example: a literature-review service



How to work

1. Run the notebook cells from top to bottom.
2. Cells marked **GAP** are yours to complete.
3. There are five checkpoints, each focusing on a different part of this service (tools, client, agent, etc.).
4. If you get stuck, mcp_literature_review_solution.ipynb has every gap filled in, but try to solve it yourself first.

Five checkpoints

Each checkpoint builds one part of the literature-review service above.

1 Build a tool
Implement a server tool (e.g. search_literature).

→

2 No language model
Use the client to list and call tools and read a resource.

→

3 Connect your agent
Let the model use the tools to answer your research question.

→

4 Errors and evidence
Handle no results, errors and check citations.

→

5 A boundary that holds
Decide what this agent may do and keep it within safe limits.

After the workshop session…

Where MCP sits



MCP: an agent uses tools and data

A2A: an agent hands work to another agent

Agent Skills and AGENTS.md: know-how and instructions

CLIs and code execution: often cheaper for coding agents with a shell

WebMCP: web pages expose tools to browser agents (still a draft)

Which interface would you choose?





One local analysis notebook: Python functions, or an existing CLI



An institutional API with an OpenAPI contract: use it directly; add a thin MCP layer when several hosts need it



A capability reused by several agent applications: MCP



Another team's autonomous agent: A2A at that boundary, **MCP for the tools inside**

Take home

- The session folder: server, notebooks
- Biomedical MCP servers: https://biocontext.ai/registry
- The specification: https://modelcontextprotocol.io
- Europe PMC query syntax

Thank you! ☺

Questions now, or later: open-llm@scilifelab.se

Please fill in the feedback form

Next: **Deploying AI Agents: Legal, Technical, and Practical Considerations. Speaker: Johan Alfredéen**